Word2vec

- ✓ vectors are **dense**
- ✓ the values will be **real-valued numbers** that can be negative
- ✓ proposed by Mikolov et al. (2013a, 2013b).
- ✓ skip-gram with negative sampling (SGNS)
- ✓ **static** embedding
- ✓ Code - https://www.geeksforgeeks.org/python-word-embedding-using-word2vec/

SGNS - Steps

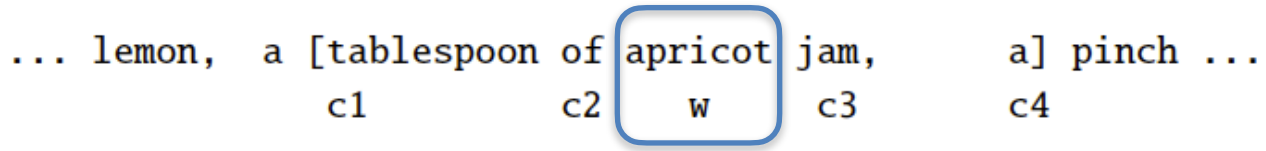
✓ Treat the target word and a neighboring context word as positive examples.

✓ Randomly sample other words in the lexicon to get negative samples.

✓Use logistic regression to train a classifier to distinguish those two cases.

✓Use the learned weights as the embeddings.

SGNS: Classifier



train a classifier such that, given a tuple (w,c) of a target word w paired with a candidate context word c

(for example (apricot, jam), or (apricot, cat))

it will return the probability that c is a real context word (true for jam, false for cat)

$P(+|w,c)$ $P(-|w,c)$

How does the classifier compute the probability ?

- ✓ base this probability on embedding similarity
- ✓ two vectors are similar if they have a high dot product (cosine similarity is just a normalized dot product)

$$Similarity(w,c) \approx \mathbf{c} \cdot \mathbf{w}$$

How does the classifier compute the probability ?

$$Similarity(w,c) \approx \mathbf{c} \cdot \mathbf{w}$$

turn the dot product into a probability, use the sigmoid function

$$\sigma(x) = \frac{1}{1+\exp(-x)}$$

$$P(+|w,c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1+\exp(-\mathbf{c} \cdot \mathbf{w})}$$

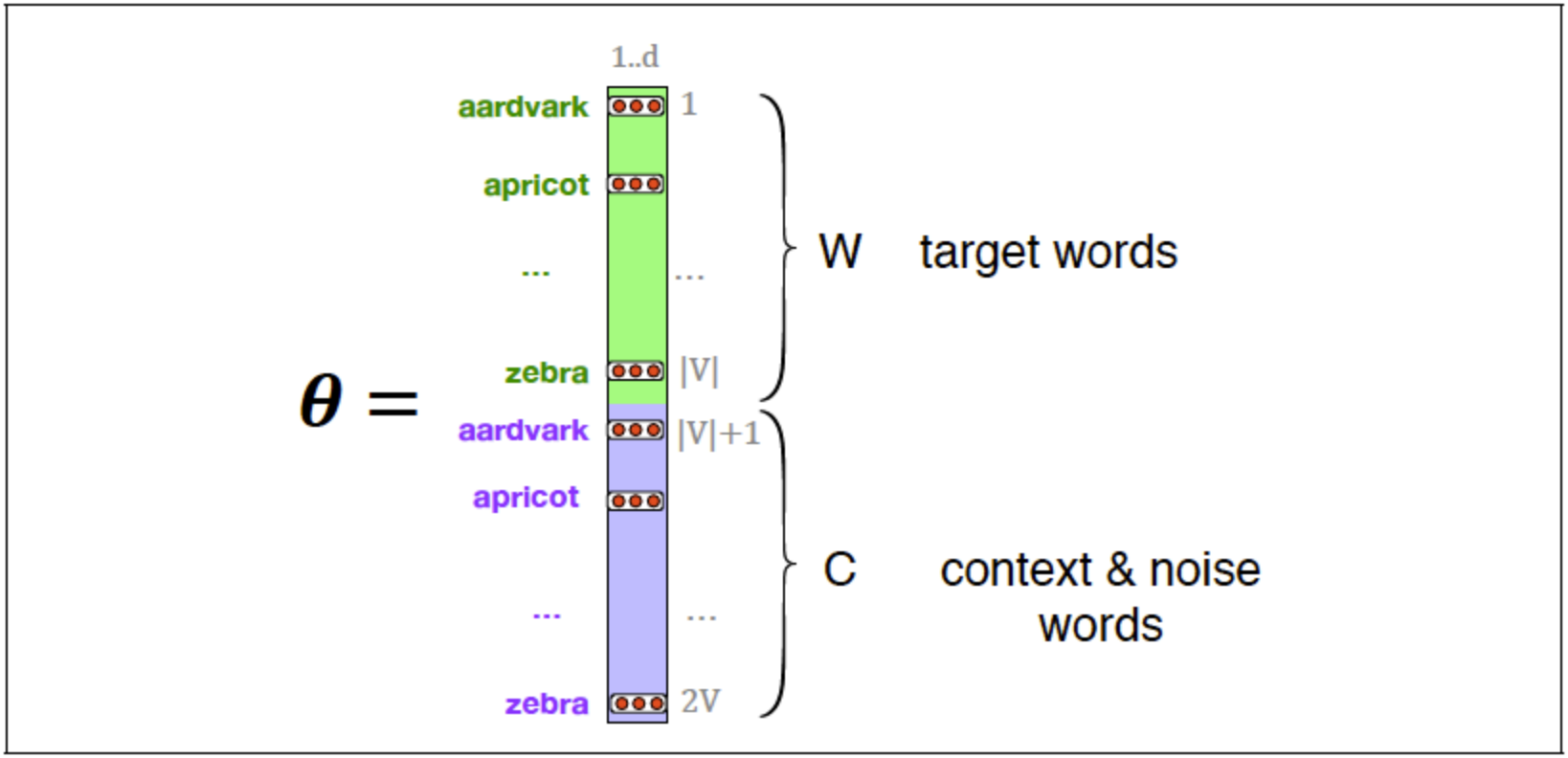
$$\begin{aligned} P(-|w,c) &= 1 - P(+|w,c) \\ &= \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1+\exp(\mathbf{c} \cdot \mathbf{w})} \end{aligned}$$

SGNS: Classifier

- ✓ *Skip-gram actually stores two embeddings for each word,*
  - ✓ *one for the word as a target*
  - ✓ *one for the word considered as context.*

- ✓ *Thus the parameters we need to learn are two matrices **W** and **C**, each containing an embedding for every one of the |**V**| words in the vocabulary **V.***

SGNS: Classifier



Learning skip-gram embeddings

- ✓ **input :** a corpus of text and a chosen vocabulary size N.
- ✓ It begins by assigning a random embedding vector for each of the N vocabulary words
- ✓ Proceeds to iteratively shift the embedding of each word w to be more like the embeddings of words that occur nearby in texts, and less like the embeddings of words that don't occur nearby.

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
              c1         c2    w     c3        c4
```

Learning skip-gram embeddings

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

*each of the (**w**, $\mathbf{c_{pos}}$) training instances, create **k** negative samples, each consisting of the target **w** plus a 'noise word' $\mathbf{c_{neg}}$*

Learning skip-gram embeddings

Given

- ✓ *the set of positive and negative training instances*
- ✓ *an initial set of embeddings*

the goal of the learning algorithm is to adjust those embeddings to

- ✓ *Maximize the similarity of the target word, context word pairs* ($\boldsymbol{w}, \boldsymbol{c}_{\boldsymbol{pos}}$) *drawn from the positive examples*

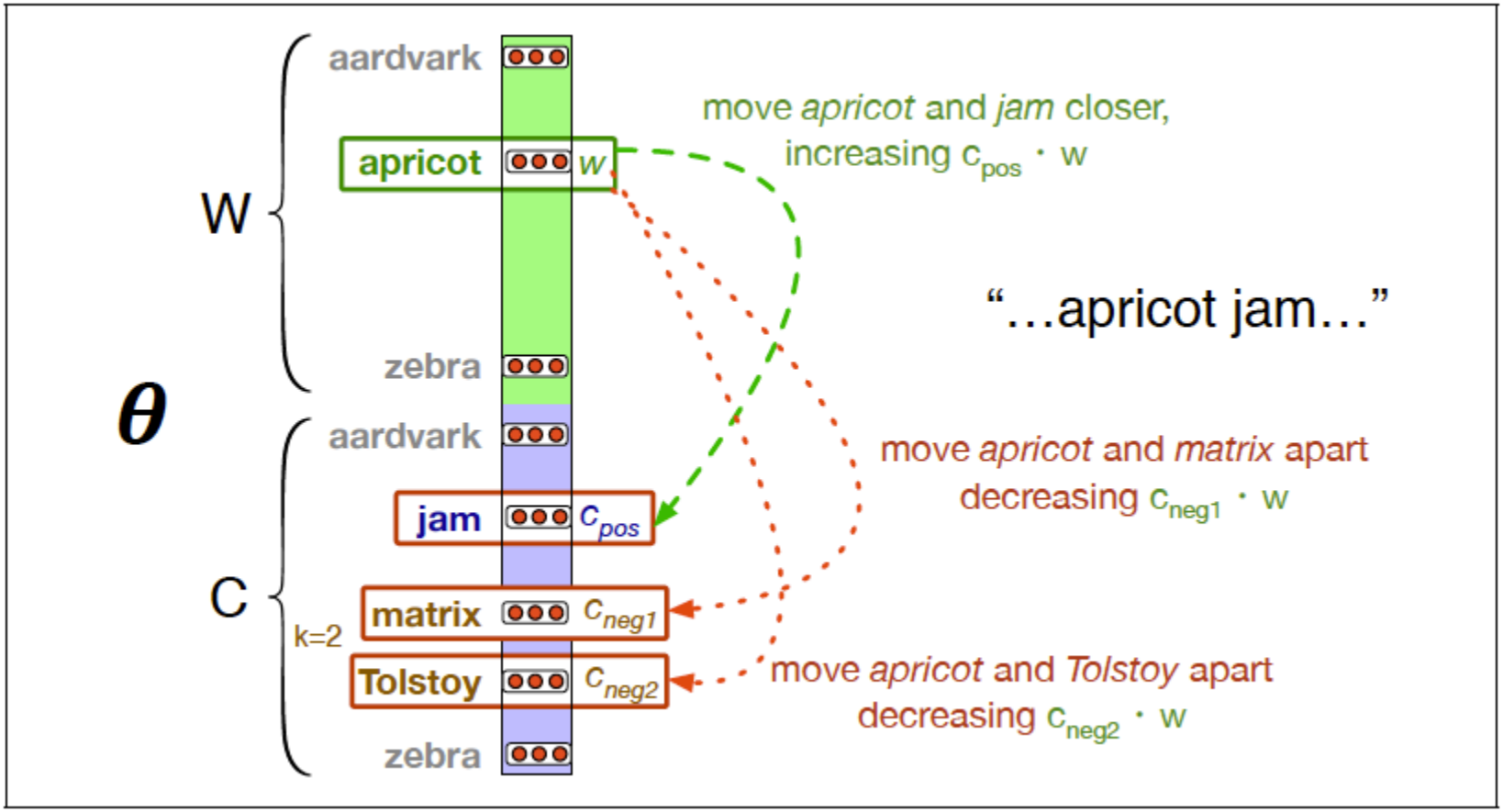
- ✓ *Minimize the similarity of the* ($\boldsymbol{w}, \boldsymbol{c}_{\boldsymbol{neg}}$) *pairs from the negative examples.*

Learning skip-gram embeddings

$$
\begin{aligned}
L_{CE} &= -\log\left[P(+|w,c_{pos})\prod_{i=1}^{k}P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log\left(1-P(+|w,c_{neg_i})\right)\right] \\
&= -\left[\log \sigma(c_{pos}\cdot w) + \sum_{i=1}^{k}\log\sigma(-c_{neg_i}\cdot w)\right]
\end{aligned}
$$

minimize this loss function using stochastic gradient descent

Learning skip-gram embeddings



Learning skip-gram embeddings

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1]\mathbf{w}$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(\mathbf{c}_{neg} \cdot \mathbf{w})]\mathbf{w}$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1]\mathbf{c}_{pos} + \sum_{i=1}^{k} [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})]\mathbf{c}_{neg_i}$$

To get the gradient, we need to take the derivative

$$\mathbf{c}_{pos}^{t+1} = \mathbf{c}_{pos}^{t} - \eta[\sigma(\mathbf{c}_{pos}^{t} \cdot \mathbf{w}^{t}) - 1]\mathbf{w}^{t}$$

$$\mathbf{c}_{neg}^{t+1} = \mathbf{c}_{neg}^{t} - \eta[\sigma(\mathbf{c}_{neg}^{t} \cdot \mathbf{w}^{t})]\mathbf{w}^{t}$$

$$\mathbf{w}^{t+1} = \mathbf{w}^{t} - \eta\left[[\sigma(\mathbf{c}_{pos}^{t} \cdot \mathbf{w}^{t}) - 1]\mathbf{c}_{pos}^{t} + \sum_{i=1}^{k} [\sigma(\mathbf{c}_{neg_i}^{t} \cdot \mathbf{w}^{t})]\mathbf{c}_{neg_i}^{t}\right]$$

update equations going from time step t to t +1 in stochastic gradient descent

Learning skip-gram embeddings

- ✓ *skip-gram model learns two separate embeddings for each word **i** :*
  - ✓ *the target embedding $\mathbf{w}_i$ and the context embedding $\mathbf{c}_i$, stored in two matrices*
- ✓ *It's common to just add them together, representing word **i** with the vector $\mathbf{w}_i+\mathbf{c}_i$.*
- ✓ *Alternatively, throw away the **C** matrix and just represent each word i by the vector $\mathbf{w}_i$.*

Semantic properties of embeddings

✓ When the vectors are computed from **short context windows**, the most similar words to a target word **w** tend to be semantically similar words with the same parts of speech.

✓ When vectors are computed from **long context windows**, the highest cosine words to a target word **w** tend to be words that are topically related but not similar.

✓ *window of 2, the most similar words to the word Hogwarts (from the Harry Potter series) were names of other fictional schools: Sunnydale (from Buffy the Vampire Slayer) or Evernight (from a vampire series).*

✓ *window of 5, the most similar words to Hogwarts were other words topically related to the Harry Potter series: Dumbledore, Malfoy, and half-blood.*

Semantic properties of embeddings

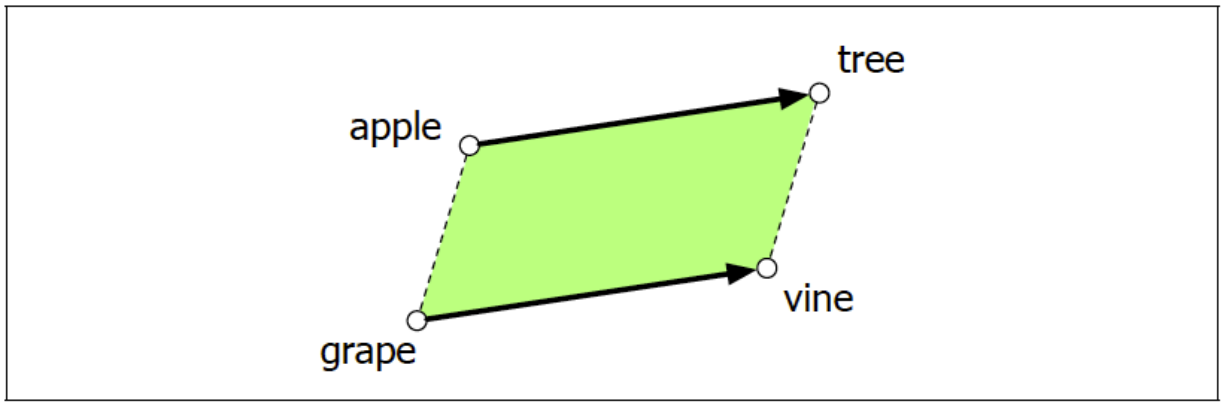
- ✓ **first-order co-occurrence**: if they are typically nearby each other.
  - ✓ ***wrote*** is a first-order associate of ***book*** or ***poem***

- ✓ **second-order co-occurrence** if they have similar neighbors.
  - ✓ ***wrote*** is a second-order associate of words like ***said*** or ***remarked***.

Semantic properties of embeddings

✓ ability to capture relational meanings.

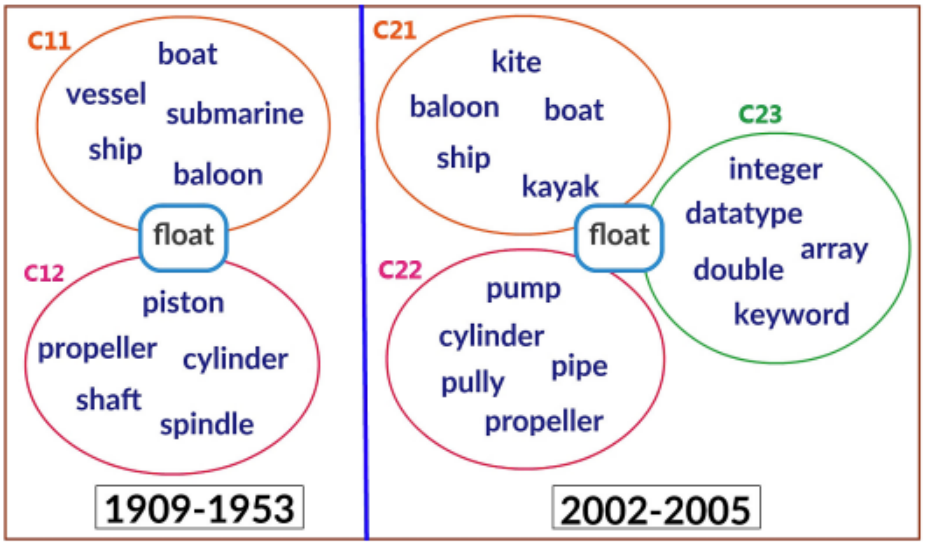
✓ ***apple:tree::grape:?,*** *i.e., apple is* to tree as grape is to _____ ??

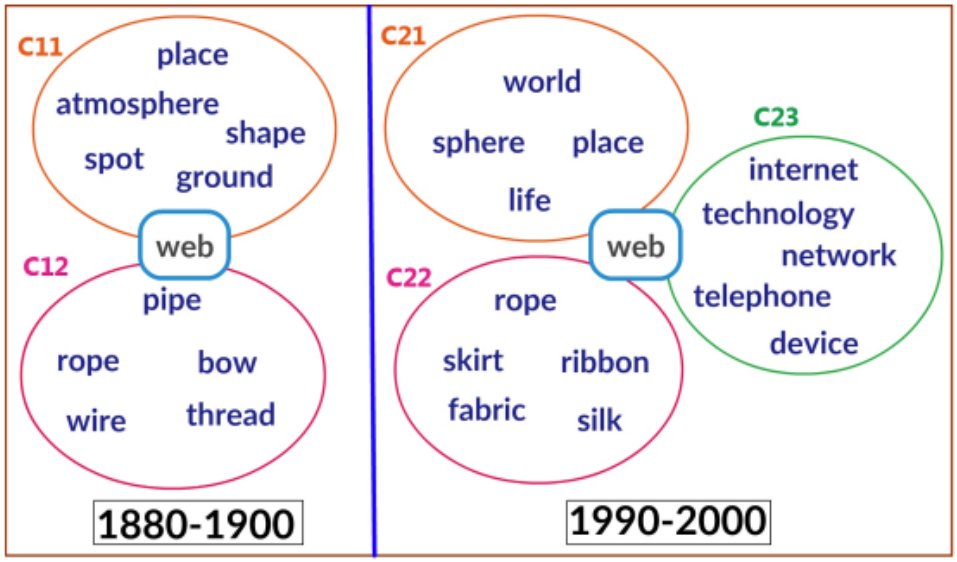


For a : b :: a : ?? problem, the parallelogram method is thus:*

$$\hat{\mathbf{b}}^* = \underset{\mathbf{x}}{\operatorname{argmin}}\ \text{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

Embeddings and Historical Semantics





Bias and Embeddings

- ✓ Man: Computer Programmer:: Woman: ??
- ✓ Father: Doctor :: Mother : ??
- ✓ **Debiasing** is still an open problem

Evaluating Vector Models

- ✓ Intrinsic Evaluation
- ✓ Extrinsic Evaluation